Memory Manager API 문서

개요

Memory Manager는 운영체제의 페이징(Paging)과 가상 메모리(Virtual Memory) 개념을 AI 에이전트의 컨텍스트 관리에 적용한 시스템입니다. 3계층 계층형 메모리 아키텍처를 통해 효율적인 메모리 관리를 제공합니다.

기본 **URL**: `http://localhost:3001/api`

---

1. 시스템 상태 (System Status)

1.1 헬스 체크

시스템의 현재 상태를 확인합니다.

```
GET /api/health
```

응답 예시:

```
{
  "status": "ok",
  "timestamp": "2025-01-25T10:30:00.000Z",
  "services": {
    "redis": "connected",
    "chromadb": "connected",
    "mongodb": "connected"
  }
}
```

1.2 통계 조회

메모리 관리자의 통계 정보를 확인합니다.

```
GET /api/stats
```

응답 예시:

```
{
  "l1Size": 45,
  "l1Capacity": 100,
  "l2Size": 234,
  "l3Size": 1523,
  "totalAccesses": 5432,
  "pageFaults": 108,
  "hitRate": 0.85,
  "averageAccessTime": 8.5,
  "evictions": 234,
  "promotions": 45,
  "demotions": 123
}
```

---

2. 메모리 연산 (Memory Operations)

2.1 값 저장 (PUT)

지정된 키로 값을 저장합니다.

```
POST /api/memory/put
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-001",
  "key": "conversation:123",
  "value": "User asked about the weather in Seoul",
  "metadata": {
    "type": "conversation",
    "timestamp": "2025-01-25T10:30:00.000Z"
  }
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |---|---|---|---| | agentId | string | Y | 에이전트 ID (메모리 격리) | | key | string | Y | 메모리 키 (페이지 식별자) | | value | string | Y | 저장할 값 (직렬화된 JSON) | | metadata | object | N | 추가 메타데이터 |

응답 예시:

```
{
  "success": true,
  "level": "L1_CACHE",
  "accessTime": 1.2,
  "pageFault": false,
  "message": "Value stored successfully in L1 cache"
}
```

2.2 값 조회 (GET)

저장된 값을 검색합니다.

```
POST /api/memory/get
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-001",
  "key": "conversation:123"
}
```

응답 예시:

```
{
  "success": true,
  "data": "User asked about the weather in Seoul",
  "level": "L1_CACHE",
  "accessTime": 1.5,
  "pageFault": false,
  "message": "Retrieved from L1 cache"
}
```

2.3 의미론적 검색 (Semantic Search)

벡터 유사도 검색을 수행합니다.

```
POST /api/memory/search
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-001",
```

```
  "query": "weather discussion",
  "topK": 5,
  "threshold": 0.7
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |——|——|——|——| | agentId | string | Y | 에이전트 ID | | query | string | Y | 검색 쿼리 | | topK | number | N | 반환할 결과 수 (기본값: 5) | | threshold | number | N | 유사도 임계값 (0-1, 기본값: 0.5) |

응답 예시:

```
{
  "success": true,
  "results": [
    {
      "key": "conversation:123",
      "value": "User asked about the weather in Seoul",
      "similarity": 0.89,
      "level": "L2_VECTOR"
    },
    {
      "key": "conversation:456",
      "value": "Discussed temperature and humidity",
      "similarity": 0.76,
      "level": "L2_VECTOR"
    }
  ],
  "accessTime": 18.5
}
```

2.4 값 삭제 (DELETE)

지정된 키의 값을 삭제합니다.

```
DELETE /api/memory
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-001",
  "key": "conversation:123"
}
```

응답 예시:

```
{
  "success": true,
  "message": "Value deleted successfully from all levels"
}
```

2.5 메모리 초기화 (CLEAR)

에이전트의 모든 메모리를 초기화합니다.

```
POST /api/memory/clear
Content-Type: application/json
```

요청 본문:

```
{
  "agentId": "agent-001"
}
```

응답 예시:

```
{
  "success": true,
  "deletedCount": 1523,
  "message": "All memory cleared for agent-001"
}
```

---

3. 계층별 동작 (Tier Behavior)

3.1 L1 캐시 (Redis)

- 접근 시간: ~1ms
- 용량: 기본 100페이지
- 교체 정책: LRU (Least Recently Used)
- **TTL**: 기본 5분 (300,000ms)

특징: - 가장 빠른 접근 - 자주 사용되는 데이터 (Hot Data) - 일시적 저장

3.2 L2 벡터 DB (ChromaDB)

- 접근 시간: ~10ms
- 용량: 무제한 (디스크 기반)
- 검색: 벡터 유사도 검색
- 임베딩: Ollama nomic-embed-text

특징: - 의미론적 검색 지원 - 중요도 중간 데이터 - 자동 승격/강등

3.3 L3 저장소 (MongoDB)

- 접근 시간: ~50ms
- 용량: 무제한
- 저장 방식: 영구 저장
- **Page Fault**: L3 접근 시 발생

특징: - 영구적 저장 - 찾기 힘든 데이터 (Cold Data) - 안전한 내구성

---

4. OS 개념 매핑 (OS Concepts Mapping)

OS 개념	구현	설명
Paging	MemoryPage	메모리 페이지를 기본 단위로 사용
Page Table	PageTableEntry	논리 주소를 물리 주소로 매핑
LRU Eviction	LRUCache	가장 오래 전에 사용된 페이지 교체
Page Fault	Page Fault 처리	L3 접근 시 자동 승격
Virtual Memory	3계층 아키텍처	계층형 메모리 계층 구현
Thrashing Prevention	통계 모니터링	액세스 패턴 추적 및 최적화

---

5. 에러 응답 (Error Responses)

5.1 에러 형식

모든 에러 응답은 다음 형식을 따릅니다.

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request parameters",
    "details": {
      "field": "agentId",
      "issue": "must be a non-empty string"
    }
  }
}
```

5.2 HTTP 상태 코드

코드	설명
200	성공
400	잘못된 요청
404	찾을 수 없음
500	서버 오류

5.3 에러 코드

코드	설명
VALIDATION_ERROR	요청 파라미터 검증 실패
AGENT_NOT_FOUND	에이전트를 찾을 수 없음
KEY_NOT_FOUND	키를 찾을 수 없음
L1_CACHE_FULL	L1 캐시가 가득 참
EMBEDDING_FAILED	임베딩 생성 실패
MONGODB_ERROR	MongoDB 오류
REDIS_ERROR	Redis 오류
CHROMADB_ERROR	ChromaDB 오류

---

6. 사용 예제 (Usage Examples)

6.1 Node.js 클라이언트

```
const axios = require('axios');

class MemoryManagerClient {
  constructor(baseURL = 'http://localhost:3001/api') {
    this.client = axios.create({ baseURL });
  }

  // 값 저장
  async put(agentId, key, value, metadata = {}) {
    const response = await this.client.post('/memory/put', {
      agentId,
```

```
      key,
      value,
      metadata
    });
    return response.data;
  }

  // 값 조회
  async get(agentId, key) {
    const response = await this.client.post('/memory/get', {
      agentId,
      key
    });
    return response.data;
  }

  // 의미론적 검색
  async search(agentId, query, options = {}) {
    const response = await this.client.post('/memory/search', {
      agentId,
      query,
      topK: options.topK || 5,
      threshold: options.threshold || 0.7
    });
    return response.data;
  }

  // 통계 조회
  async getStats() {
    const response = await this.client.get('/stats');
    return response.data;
  }
}

// 사용 예시
const client = new MemoryManagerClient();

async function main() {
  try {
    // 값 저장
    await client.put(
      'agent-001',
      'conversation:123',
      'User asked about AI',
      { type: 'conversation' }
    );

    // 값 조회
    const result = await client.get('agent-001', 'conversation:123');
    console.log('Retrieved:', result.data);

    // 의미론적 검색
    const searchResults = await client.search('agent-001', 'artificial
intelligence');
    console.log('Search results:', searchResults.results);
```

```
    // 통계
    const stats = await client.getStats();
    console.log('Hit rate:', stats.hitRate);

  } catch (error) {
    console.error('Error:', error.response?.data || error.message);
  }
}

main();
```

6.2 Python 클라이언트

```
import requests
from typing import Optional, Dict, Any, List

class MemoryManagerClient:
    def __init__(self, base_url: str = "http://localhost:3001/api"):
        self.base_url = base_url

    def put(
        self,
        agent_id: str,
        key: str,
        value: str,
        metadata: Optional[Dict[str, Any]] = None
    ) -> Dict[str, Any]:
        """값 저장"""
        response = requests.post(
            f"{self.base_url}/memory/put",
            json={
                "agentId": agent_id,
                "key": key,
                "value": value,
                "metadata": metadata or {}
            }
        )
        response.raise_for_status()
        return response.json()

    def get(self, agent_id: str, key: str) -> Dict[str, Any]:
        """값 조회"""
        response = requests.post(
            f"{self.base_url}/memory/get",
            json={
                "agentId": agent_id,
                "key": key
            }
        )
        response.raise_for_status()
        return response.json()

    def search(
        self,
        agent_id: str,
        query: str,
        top_k: int = 5,
```

```python
        threshold: float = 0.7
    ) -> Dict[str, Any]:
        """의미론적 검색"""
        response = requests.post(
            f"{self.base_url}/memory/search",
            json={
                "agentId": agent_id,
                "query": query,
                "topK": top_k,
                "threshold": threshold
            }
        )
        response.raise_for_status()
        return response.json()

    def get_stats(self) -> Dict[str, Any]:
        """통계 조회"""
        response = requests.get(f"{self.base_url}/stats")
        response.raise_for_status()
        return response.json()

# 사용 예시
client = MemoryManagerClient()

try:
    # 값 저장
    client.put(
        "agent-001",
        "conversation:123",
        "User asked about AI",
        {"type": "conversation"}
    )

    # 값 조회
    result = client.get("agent-001", "conversation:123")
    print(f"Retrieved: {result['data']}")

    # 의미론적 검색
    search_results = client.search("agent-001", "artificial intelligence")
    print(f"Search results: {len(search_results['results'])} found")

    # 통계
    stats = client.get_stats()
    print(f"Hit rate: {stats['hitRate']:.2%}")

except Exception as e:
    print(f"Error: {e}")
```

---

7. 성능 최적화 (Performance Optimization)

7.1 캐시 히트율 최적화

```javascript
// 자주 사용되는 데이터를 L1에 유지
async function maintainHotData(client, agentId) {
  const hotKeys = ['config', 'user:profile', 'session:data'];
```

```
  for (const key of hotKeys) {
    // 주기적으로 접근하여 L1 유지
    await client.get(agentId, key);
  }
}
```

7.2 대량 쓰기 최적화

```
// 배치 처리로 오버헤드 감소
async function batchPut(client, agentId, items) {
  const promises = items.map(item =>
    client.put(agentId, item.key, item.value, item.metadata)
  );
  await Promise.all(promises);
}
```

7.3 의미론적 검색 최적화

```
// 적절한 topK와 threshold 설정
async function optimizedSearch(client, agentId, query) {
  // 높은 임계값으로 정확도 향상
  const results = await client.search(agentId, query, {
    topK: 3,       // 상위 3개 결과만
    threshold: 0.8  // 높은 유사도
  });

  return results.results;
}
```

---

8. 모니터링 (Monitoring)

8.1 메트릭

메트릭	설명	목표값
hitRate	캐시 히트율	> 80%
averageAccessTime	평균 접근 시간	< 10ms
pageFaultRate	페이지 부재율	< 20%
l1Size	L1 현재 크기	< l1Capacity
evictions	페이지 교체 횟수	모니터링

8.2 상태 확인

```
# 통계 조회
curl http://localhost:3001/api/stats

# 헬스 체크
curl http://localhost:3001/api/health
```

---

9. 구성 (Configuration)

환경 변수 (.env):

```
# L1 캐시 (Redis)
REDIS_HOST=localhost
REDIS_PORT=6379
L1_CACHE_SIZE=100
```

```
L1_TTL=300000  # 5분

# L2 벡터 DB (ChromaDB)
CHROMADB_HOST=localhost
CHROMADB_PORT=8000

# L3 저장소 (MongoDB)
MONGODB_URI=mongodb://localhost:27017
MONGODB_DB_NAME=memory_manager

# 임베딩 (Ollama)
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_EMBEDDING_MODEL=nomic-embed-text
```

---

10. 변경 로그 (Changelog)

v1.0.0 (2025-01-25)

- 초기 릴리스
- 3계층 메모리 계층 구조
- LRU 캐시 교체 정책
- 의미론적 검색 지원
- 94.44% 테스트 커버리지

문서 버전: 1.0.0 최종 업데이트: 2025-01-25 연락처: GitHub Issues